UNIVERSIDADE ESTADUAL DO MARANHÃO
CENTRO DE CIÊNCIAS TECNOLÓGICAS
NÚCLEO DE TECNOLOGIAS PARA EDUCAÇÃO
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ANDRÉ PEREIRA FERREIRA

ANÁLISE COMPARATIVA DE DESEMPENHO EM AMBIENTES DE PERSISTÊNCIA EM MEMÓRIA: o impacto da arquitetura Worker do FrankenPHP no framework CodeIgniter 4 frente ao ecossistema Laravel Octane

Disciplina: Trabalho de Conclusão de Curso (TCC)
Atividade: Orientação – Estrutura Inicial do Trabalho

Grajaú – MA, 2026

ANDRÉ PEREIRA FERREIRA

ANÁLISE COMPARATIVA DE DESEMPENHO EM AMBIENTES DE PERSISTÊNCIA EM MEMÓRIA: o impacto da arquitetura Worker do FrankenPHP no framework CodeIgniter 4 frente ao ecossistema Laravel Octane

Trabalho apresentado à disciplina de Trabalho de Conclusão de Curso (TCC), do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Estadual do Maranhão, como requisito parcial da atividade Orientação – Estrutura Inicial do Trabalho.

Orientador(a): [Titulação e nome do(a) orientador(a) — a definir].

Grajaú – MA

2026

RESUMO

Este trabalho analisa o impacto do uso de servidores de aplicação baseados em processos persistentes na performance do framework CodeIgniter 4 (CI4). O estudo compara o modelo tradicional PHP-FPM com o FrankenPHP operando em dois modos — Clássico e Worker (este último não suportado nativamente pelo CI4, exigindo uma solução de integração customizada) —, utilizando o Laravel Octane como referência de mercado para arquiteturas de persistência em memória. A pesquisa caracteriza-se como experimental, quantitativa e comparativa: os quatro cenários são executados em contêineres Docker com limites equivalentes de CPU e memória, e submetidos a testes de carga com o Grafana k6, avaliando vazão (RPS), latência, taxa de erro e consumo de recursos computacionais. Nesta etapa, apresenta-se a estrutura inicial do trabalho — elementos pré-textuais, introdução com problema, justificativa, objetivos e questão norteadora, fundamentação teórica e metodologia —, ainda sem os resultados experimentais, que serão consolidados nas etapas seguintes da pesquisa. Espera-se, ao final, avaliar se a arquitetura de persistência em memória justifica a complexidade de implementação em frameworks legados ou em aplicações monolíticas que utilizam o CI4 no backend.

Palavras-chave: CodeIgniter 4; FrankenPHP; Desempenho; Benchmarking; PHP assíncrono.

ABSTRACT

This work analyzes the impact of using persistent-process application servers on the performance of the CodeIgniter 4 (CI4) framework. The study compares the traditional PHP-FPM model with FrankenPHP operating in two modes — Classic and Worker (the latter not natively supported by CI4, requiring a custom integration solution) —, using Laravel Octane as a market reference for in-memory persistence architectures. The research is characterized as experimental, quantitative and comparative: the four scenarios run in Docker containers with equivalent CPU and memory limits and are submitted to load tests with Grafana k6, evaluating throughput (RPS), latency, error rate and resource consumption. At this stage, the initial structure of the work is presented — pre-textual elements, introduction with problem statement, justification, objectives and guiding question, theoretical framework and methodology —, still without the experimental results, which will be consolidated in the following stages of the research. Ultimately, the study aims to assess whether the in-memory persistence architecture justifies the implementation complexity in legacy frameworks or monolithic applications that use CI4 in the backend.

Keywords: CodeIgniter 4; FrankenPHP; Performance; Benchmarking; Asynchronous PHP.

LISTA DE QUADROS

Quadro 1 Comparação conceitual entre arquiteturas de execução PHP 10

Quadro 2 Cenários experimentais e alocação de recursos 12

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
CI4	CodeIgniter 4
CPU	Unidade Central de Processamento (Central Processing Unit)
PHP-FPM	Servidor web tradicional (FastCGI Process Manager)
RAM	Memória de Acesso Aleatório (Random Access Memory)
RPS	Requisições por Segundo (Requests Per Second)
TCC	Trabalho de Conclusão de Curso
UEMA	Universidade Estadual do Maranhão
VUs	Usuários Virtuais (Virtual Users)

SUMÁRIO

1 INTRODUÇÃO

1.1 Objetivo Geral

1.2 Objetivos Específicos

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Avaliação de desempenho de sistemas computacionais

2.2 A arquitetura tradicional do PHP: modelo Request-Response-Die e o PHP-FPM

2.3 Servidores de aplicação e persistência em memória no PHP

2.4 O ecossistema FrankenPHP e a arquitetura Worker

2.5 Tecnologias correlatas de persistência em memória: Swoole/OpenSwoole e RoadRunner

2.6 Instrumentação para avaliação de desempenho: Grafana k6 e contêineres Docker

2.7 Trabalhos correlatos

3 METODOLOGIA

3.1 Caracterização da pesquisa

3.2 Ambiente experimental

3.3 Instrumentação e geração de carga

3.4 Variáveis e métricas analisadas

3.5 Procedimentos de análise

REFERÊNCIAS

1 INTRODUÇÃO

A evolução das aplicações web tem ampliado a necessidade de arquiteturas capazes de oferecer maior desempenho, escalabilidade e menor tempo de resposta diante do crescimento da demanda por serviços digitais. Nesse contexto, as decisões relacionadas ao modelo de execução das aplicações assumem papel relevante, especialmente em sistemas desenvolvidos com a linguagem PHP, amplamente utilizada no desenvolvimento de aplicações web. Tradicionalmente, aplicações PHP são executadas seguindo um modelo no qual cada requisição HTTP é processada de forma independente, sendo comum a utilização do PHP-FPM (FastCGI Process Manager) para gerenciamento dos processos responsáveis pela execução da aplicação. Embora esse modelo apresente ampla maturidade e utilização em ambientes de produção, parte do custo computacional associado à inicialização do framework e à preparação do ambiente de execução ocorre repetidamente a cada requisição.

Com o avanço dos servidores de aplicação modernos, novas possibilidades de execução de aplicações PHP passaram a explorar a permanência da aplicação em memória. Tecnologias como o Swoole e o FrankenPHP possibilitam modelos de execução baseados em processos persistentes, reduzindo a necessidade de repetir etapas de inicialização da aplicação a cada requisição. No ecossistema Laravel, essa abordagem é utilizada pelo Laravel Octane, que permite a execução da aplicação sobre servidores de aplicação de longa duração. Dessa forma, torna-se relevante investigar se os benefícios proporcionados pela persistência em memória também podem ser obtidos em outros frameworks PHP, especialmente em aplicações desenvolvidas com CodeIgniter 4.

O CodeIgniter 4 (CI4) caracteriza-se por ser um framework PHP de estrutura relativamente leve e utilizado no desenvolvimento de aplicações web. Entretanto, diferentemente do ecossistema Laravel com Octane, o CI4 não oferece suporte nativo ao modo Worker do FrankenPHP. Essa limitação cria uma questão relevante para aplicações existentes que utilizam o framework: seria possível obter ganhos significativos de desempenho por meio da adoção do FrankenPHP em modo Worker, mesmo com a necessidade de mecanismos adicionais para manter a aplicação em memória? Além disso, é necessário verificar se a utilização do FrankenPHP em seu modo Clássico apresenta vantagens suficientes em relação ao modelo tradicional baseado em PHP-FPM para justificar sua adoção.

Diante desse cenário, esta pesquisa é direcionada à análise comparativa do desempenho do CodeIgniter 4 submetido a diferentes arquiteturas de execução. São considerados o modelo tradicional baseado em PHP-FPM, o FrankenPHP em modo Clássico e o FrankenPHP em modo Worker, sendo este último implementado mediante uma solução de integração que possibilite a permanência da aplicação em memória. Como referência adicional, é utilizado o Laravel com Laravel Octane, permitindo estabelecer um parâmetro de comparação com uma solução já consolidada no ecossistema PHP.

A questão que orienta esta investigação pode ser formulada da seguinte maneira: em que medida a adoção de arquiteturas de persistência em memória — em particular o FrankenPHP em modo Worker — melhora o desempenho de aplicações desenvolvidas com CodeIgniter 4 em relação ao modelo tradicional baseado em PHP-FPM, e como esse ganho se compara ao obtido pelo Laravel Octane, solução já nativamente preparada para esse modelo de execução? Parte-se da hipótese de que a persistência em memória proporciona ganhos relevantes de vazão e latência para o CodeIgniter 4, ainda que em magnitude inferior à observada no Laravel Octane, dado que este último conta com suporte nativo ao ciclo de vida de aplicações de longa duração, enquanto o CI4 depende de soluções de integração customizadas.

A relevância da pesquisa está relacionada à necessidade de obtenção de dados experimentais que auxiliem na tomada de decisões sobre modernização e escalabilidade de aplicações PHP. A identificação de possíveis ganhos de desempenho pode contribuir tanto para novos projetos quanto para aplicações existentes que utilizam CodeIgniter 4 e que necessitam aumentar sua capacidade de processamento sem necessariamente realizar uma migração completa para outro framework ou linguagem. Além disso, a avaliação do consumo de recursos pode contribuir para a análise da relação entre desempenho e utilização de infraestrutura computacional, o que é particularmente relevante em cenários de nuvem, nos quais o consumo de CPU e memória impacta diretamente os custos operacionais. Do ponto de vista acadêmico, a pesquisa também se justifica pela escassez relativa de literatura científica que compare especificamente CodeIgniter 4, PHP-FPM e FrankenPHP em modo Worker — ainda que exista farta documentação técnica e literatura consolidada sobre avaliação de desempenho de sistemas computacionais de forma geral, conforme discutido na Fundamentação Teórica.

Para realizar essa investigação, é conduzido um estudo experimental e quantitativo, utilizando ambientes controlados e equivalentes em termos de recursos computacionais. As aplicações são executadas em contêineres Docker e submetidas a testes de carga utilizando o Grafana k6, enquanto métricas relacionadas ao desempenho e ao consumo de recursos são coletadas para posterior análise. São consideradas, principalmente, métricas de vazão, expressa em requisições por segundo (RPS), latência e utilização de recursos computacionais.

Assim, a pesquisa busca produzir evidências experimentais sobre o impacto da persistência em memória no desempenho de aplicações desenvolvidas com CodeIgniter 4, contribuindo para a compreensão das possibilidades de utilização do FrankenPHP em diferentes modos de operação e estabelecendo uma comparação com arquiteturas tradicionais e com uma solução consolidada no ecossistema Laravel.

1.1 Objetivo Geral

Analisar comparativamente o desempenho do framework CodeIgniter 4 quando executado sob diferentes arquiteturas de servidores de aplicação PHP, com ênfase nos efeitos da persistência em memória proporcionada pelo FrankenPHP em modo Worker, utilizando o PHP-FPM e o FrankenPHP em modo Clássico como cenários de comparação e o Laravel Octane como referência de desempenho.

1.2 Objetivos Específicos

- Descrever as características dos modelos de execução PHP-FPM, FrankenPHP em modo Clássico e FrankenPHP em modo Worker, considerando seus mecanismos de processamento e gerenciamento da aplicação em memória;
- Implementar ambientes experimentais controlados para a execução do CodeIgniter 4 nas diferentes arquiteturas analisadas, mantendo equivalência nas condições de hardware e infraestrutura;
- Avaliar o desempenho das arquiteturas por meio de testes de carga, considerando métricas como vazão, expressa em requisições por segundo (RPS), e latência;
- Comparar os resultados obtidos pelo CodeIgniter 4 com PHP-FPM, FrankenPHP Clássico e FrankenPHP Worker, utilizando o Laravel Octane como referência adicional de desempenho;
- Analisar a relação entre os ganhos de desempenho observados, o consumo de recursos computacionais e a complexidade necessária para adoção de arquiteturas baseadas em persistência de memória.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Avaliação de desempenho de sistemas computacionais

A análise de desempenho em Engenharia de Software requer uma metodologia rigorosa de medição empírica e controle do ambiente experimental (JAIN, 1991). Segundo Jain (1991), a caracterização de desempenho deve considerar métricas fundamentais como a vazão (*throughput*), medida em requisições por segundo (RPS), e a latência, que compreende o tempo total de resposta sob diferentes níveis de estresse; o autor também discute a seleção de métricas, os tipos de carga de trabalho (*workload*), as técnicas de medição e as fontes de erro comuns em experimentos de desempenho, elementos que orientam diretamente o desenho experimental adotado nesta pesquisa (ver Metodologia).

Benchmarking, entretanto, não se resume à execução de um teste e à observação de um número isolado. Zhan et al. (2022) argumentam que a definição do problema, a instanciação do sistema sob teste, a caracterização da carga de trabalho e o próprio processo de medição influenciam diretamente o resultado obtido, de modo que um benchmark cientificamente válido exige que essas escolhas sejam explicitadas e justificadas. Essa perspectiva sustenta o cuidado desta pesquisa em manter controlados os limites de CPU e memória, as versões de software, o ambiente de execução em contêineres, o perfil de carga e a duração dos testes entre os diferentes cenários avaliados.

A busca por comparabilidade entre resultados experimentais também é discutida por Brunst et al. (2022), que relatam desafios enfrentados na aplicação de um benchmark padronizado (SPEChpc 2021) em diferentes sistemas de computação de alto desempenho. Embora tratem de um domínio distinto — arquiteturas HPC —, os autores reforçam a importância de um processo de avaliação justo e reprodutível, no qual variações de ambiente e configuração sejam minimizadas para que as diferenças observadas possam ser atribuídas à variável de interesse (no caso desta pesquisa, a arquitetura do servidor de aplicação).

Por fim, Jia et al. (2013) demonstram experimentalmente que a variedade de aplicações e a escala dos conjuntos de dados utilizados em um benchmark afetam as tendências de desempenho observadas, de modo que resultados obtidos com uma carga de trabalho específica não devem ser generalizados de forma automática para outros perfis de uso. Essa constatação é considerada nas limitações desta pesquisa: os resultados obtidos estarão associados ao perfil de carga definido na Metodologia (usuários virtuais e padrão de requisições do CodeIgniter 4), não devendo ser extrapolados, sem ressalvas, para aplicações com características distintas de I/O ou de acesso a banco de dados.

Um ponto adicional, e diretamente relevante ao rigor experimental buscado neste trabalho, é discutido por Laaber, Basmaci e Salza (2021): os autores demonstram, em um experimento de larga escala com mais de 4 mil benchmarks de software, que a instabilidade de um benchmark — isto é, a alta variabilidade entre medições repetidas de uma mesma

métrica — compromete a confiabilidade da avaliação de desempenho e pode ser influenciada por características do próprio código avaliado e do ambiente de execução. Os autores reforçam ainda que aumentar o número de repetições de medição tende a produzir estimativas mais estáveis. Essa constatação sustenta, nesta pesquisa, tanto o isolamento rigoroso do ambiente (contêineres com recursos limitados, limpeza de cache do sistema operacional entre execuções) quanto a necessidade de tratar eventuais variações de vazão e latência entre execuções de um mesmo cenário como parte esperada da variabilidade experimental, e não como erro de medição a ser ignorado.

2.2 A arquitetura tradicional do PHP: modelo Request-Response-Die e o PHP-FPM

A arquitetura histórica de execução da linguagem PHP fundamenta-se no modelo *share-nothing* (sem compartilhamento de estado), no qual cada requisição HTTP recebida pelo servidor resulta na inicialização do interpretador, no carregamento dos arquivos da aplicação, na execução das instruções e na imediata destruição de todos os recursos da memória ao término da resposta (LOCKHART, 2015).

Com a consolidação das aplicações web modernas, o gerenciamento de processos evoluiu com a adoção do PHP-FPM (*FastCGI Process Manager*), documentado oficialmente como uma implementação de FastCGI voltada a sites de alto tráfego, que mantém grupos (*pools*) de processos que podem ser configurados com diferentes usuários, diretórios e arquivos de configuração (PHP DOCUMENTATION GROUP, 2024). O PHP-FPM oferece três modos de gerenciamento de processos filhos — `static`, um número fixo de processos; `dynamic`, processos criados sob demanda até um limite máximo; e `ondemand`, processos criados apenas quando chegam requisições — controlados, entre outros parâmetros, por `pm.max_children` (PHP DOCUMENTATION GROUP, 2024). Ainda que essa abordagem reduza a sobrecarga do sistema operacional ao evitar reinvocar o binário do PHP a cada chamada, o modelo tradicional sob PHP-FPM ainda impõe que frameworks de maior porte, como o CodeIgniter 4, realizem o ciclo completo de *bootstrapping* — carregamento de rotas, dependências e configurações — a cada nova requisição (LOCKHART, 2015).

2.3 Servidores de aplicação e persistência em memória no PHP

Para superar os gargalos associados ao *bootstrapping* repetitivo, surgiram alternativas baseadas em loops de eventos e servidores de aplicação de longa duração. Nesse paradigma, a aplicação é inicializada uma única vez e mantida residente na memória RAM, permanecendo pronta para processar múltiplas requisições concorrentes sem recomputar a fase de *boot*.

No ecossistema Laravel, essa abordagem foi padronizada por meio do Laravel Octane, que atua como uma camada de abstração capaz de servir a aplicação sobre diferentes servidores de aplicação — FrankenPHP, Open Swoole, Swoole e RoadRunner — mantendo-a carregada em memória entre requisições (OTWELL, 2024). A documentação oficial do Octane é explícita quanto aos riscos inerentes a esse modelo: como a mesma instância da aplicação é reutilizada em requisições sucessivas, dados armazenados de forma estática (por exemplo, em um array estático de uma classe) tendem a se acumular indefinidamente, configurando vazamento de memória (*memory leak*), e o estado de serviços registrados como singletons pode "vazar" de uma requisição para outra caso o container de injeção de dependências, a requisição HTTP ou o repositório de configuração sejam injetados diretamente no construtor de outros objetos (OTWELL, 2024). Como mitigação, o Octane reinicia periodicamente cada *worker* após um número configurável de requisições processadas — por padrão, 500, ajustável pela flag `--max-requests` — de modo a conter vazamentos de memória não tratados pela aplicação (OTWELL, 2024).

Por outro lado, frameworks mais leves e tradicionais, como o CodeIgniter 4 (CI4), não contam com suporte nativo ou ecossistema integrado equivalente ao Octane para execução em loops persistentes, sendo tradicionalmente projetados para o ciclo de vida dinâmico do PHP-FPM (CODEIGNITER FOUNDATION, 2024). Isso significa que, para operar em modo Worker, o CI4 depende da criação de uma solução de integração (*bridge*) responsável por reinicializar o estado da aplicação a cada requisição dentro do mesmo processo persistente — o problema central investigado por esta pesquisa.

2.4 O ecossistema FrankenPHP e a arquitetura Worker

O FrankenPHP surge como um servidor de aplicação moderno para PHP, escrito em Go (Golang) e construído sobre o servidor web Caddy (DUNGALAS, 2024; CADDY SERVER, 2024). O diferencial do FrankenPHP reside na integração direta do interpretador PHP (*libphp*) com o servidor web por meio de *threads* em Go via CGO, eliminando a dependência do protocolo FastCGI típica do PHP-FPM tradicional (DUNGALAS, 2024).

O FrankenPHP opera sob dois modos principais de execução (DUNGALAS, 2024):

- **Modo Clássico**: sem configuração adicional, o FrankenPHP funciona como um substituto direto e transparente para o PHP-FPM ou para o Apache com mod_php, executando cada script individualmente a cada requisição. De forma semelhante ao Caddy, aceita um número ilimitado de conexões, mas as atende com um número fixo de *threads* PHP inicializadas na *startup* — comportamento comparável ao modo `static` do PHP-FPM —, podendo também ser configurado para escalar automaticamente em tempo de execução, de modo análogo ao modo `dynamic` (DUNGALAS, 2024).

- **Modo Worker**: a aplicação é carregada uma única vez na memória durante o processamento inicial e passa a atender as requisições subsequentes em um laço contínuo de alta velocidade, sem repetir o *bootstrapping* do framework (DUNGALAS, 2024).

Um ponto central para esta pesquisa é que o Worker Mode não é apenas "mais rápido": ele altera o ciclo de vida da aplicação. A própria documentação oficial adverte que variáveis estáticas, propriedades estáticas de classes, variáveis globais e caches em memória persistem entre requisições atendidas pelo mesmo *worker* — inclusive alterações feitas em superglobais como `$_ENV` durante uma requisição permanecem visíveis nas requisições seguintes tratadas pela mesma *thread* (DUNGALAS, 2024). Enquanto frameworks como Symfony e o próprio Laravel (via Octane) já possuem suporte nativo para resetar esse estado entre requisições, sua aplicação em ecossistemas como o CodeIgniter 4 exige a implementação de soluções de integração customizadas para resetar estados e adaptar o ciclo de vida das requisições (DUNGALAS, 2024; CODEIGNITER FOUNDATION, 2024) — exatamente a lacuna técnica que esta pesquisa se propõe a explorar experimentalmente.

2.5 Tecnologias correlatas de persistência em memória: Swoole/OpenSwoole e RoadRunner

Além do FrankenPHP, outras tecnologias consolidaram o paradigma de persistência em memória no ecossistema PHP e servem de referência conceitual para este trabalho.

O OpenSwoole apresenta-se como um motor de execução assíncrona e orientada a eventos para PHP, cujo modelo de concorrência é sustentado por corrotinas (*coroutines*): unidades de execução leves, criadas por requisição, que permitem que o processamento de uma requisição avance enquanto outra aguarda uma operação de I/O, sem exigir a criação de novas *threads* ou processos do sistema operacional (OPENSWOOLE, 2024). Essa característica é relevante para a delimitação conceitual desta pesquisa: o modo Worker do FrankenPHP garante a *persistência da aplicação em memória*, mas não implementa, por si só, um modelo de concorrência baseado em corrotinas — de modo que "Worker Mode" e "PHP assíncrono" não devem ser tratados como sinônimos, ainda que o título deste trabalho faça referência a esse contexto mais amplo de PHP assíncrono/persistente.

Já o RoadRunner apresenta-se como um servidor de aplicação e gerenciador de processos PHP escrito em Go, no qual aplicações PHP são executadas como *workers* de longa duração administrados por um *pool* dedicado (ROADRUNNER, 2024). Esse *pool* de *workers* é responsável pela alocação, liberação, supervisão e destruição controlada dos processos PHP, incluindo mecanismos de reinicialização por tempo de vida (TTL) e por uso de memória

(ROADRUNNER, 2024) — um modelo de gerenciamento de ciclo de vida análogo, em objetivo, ao que este trabalho precisa implementar manualmente para viabilizar o modo Worker do FrankenPHP no CodeIgniter 4.

Comparativamente, é possível sintetizar os modelos de concorrência/persistência das tecnologias discutidas nesta pesquisa (Quadro 1).

Quadro 1 – Comparação conceitual entre arquiteturas de execução PHP

Tecnologia	Modelo de execução
PHP-FPM	Pool de processos, sem persistência da aplicação entre requisições
FrankenPHP (modo Clássico)	Pool fixo de threads PHP, sem persistência da aplicação
FrankenPHP (modo Worker)	Aplicação persistente em memória, atendida por thread(s) worker
RoadRunner	Pool de processos PHP de longa duração, gerenciados por servidor em Go
Swoole/OpenSwoole	Workers persistentes com loop de eventos e corrotinas
Laravel Octane	Camada de abstração que mantém a aplicação persistente sobre um dos servidores acima

Elaborado pelo autor (2026), com base em Dungalas (2024), Openswoole (2024), Roadrunner (2024) e Otwell (2024).

2.6 Instrumentação para avaliação de desempenho: Grafana k6 e contêineres Docker

Para submeter os ambientes a condições controladas de carga, são utilizadas ferramentas especializadas de *load testing* como o Grafana k6 (GRAFANA LABS, 2024). O k6 mede o comportamento do sistema sob teste por meio de métricas embutidas de diferentes tipos — contadores, medidores (*gauges*), taxas (*rates*) e tendências (*trends*) —, das quais `http_reqs` (volume de requisições), `http_req_duration` (latência) e `http_req_failed` (taxa de erro) são destacadas pela documentação oficial como o conjunto mínimo recomendado para análise de desempenho, por corresponderem ao método RED (*requests, errors, duration*) (GRAFANA LABS, 2024). A execução de testes de estresse com perfis de usuários virtuais (VUs) e patamares sustentados de carga permite avaliar não apenas o pico de capacidade do sistema, mas também o consumo de recursos computacionais em cada arquitetura testada (GRAFANA LABS, 2024; JAIN, 1991).

Para garantir a reprodutibilidade e o isolamento do ambiente experimental, este trabalho utiliza contêineres Docker com limites explícitos de CPU e memória, definidos por meio da especificação `deploy.resources.limits` do Docker Compose, mecanismo pelo qual é possível restringir a fração de núcleos de CPU disponíveis a um contêiner e o teto

de memória RAM que ele pode alocar antes de ser finalizado pelo *out-of-memory killer* do sistema operacional (DOCKER INC., 2024). Esse controle de recursos é o que viabiliza a equivalência de condições entre os quatro cenários experimentais descritos na Metodologia.

2.7 Trabalhos correlatos

A literatura acadêmica revisada por pares comparando especificamente CodeIgniter 4, PHP-FPM e FrankenPHP em modo Worker mostrou-se escassa no levantamento realizado para esta pesquisa. Ainda assim, foram localizados dois trabalhos acadêmicos recentes que dialogam diretamente com o problema investigado, além de benchmarks técnicos de caráter não acadêmico que ajudam a situar as expectativas deste trabalho.

O mais diretamente relacionado é o de Azzahidi, Wijayanto e Darmawan (2025), publicado no *Jurnal Teknik Informatika* (JUTIF), que avalia comparativamente o desempenho de cinco *frameworks* backend para APIs REST — Spring Boot, Flask, Express.js, Gin e **Laravel com FrankenPHP** —, medindo tempo de resposta, vazão e consumo de CPU/memória sob diferentes métodos HTTP (GET, POST, PUT, DELETE) e volumes de dados. Os autores constatam que nenhum *framework* é superior em todos os cenários: o Spring Boot dominou os testes de menor volume de dados, mas colapsou com *error rate* de 99% ao processar 1 milhão de registros; o Laravel com FrankenPHP, por sua vez, destacou-se justamente pela eficiência de memória — "a inovação [do FrankenPHP] conseguiu mitigar a fraqueza histórica do PHP em gerenciamento de memória e velocidade" (AZZAHIDI; WIJAYANTO; DARMAWAN, 2025, tradução nossa) —, ainda que sem liderar isoladamente em vazão bruta. Esse resultado é relevante para esta pesquisa por confirmar, em ambiente acadêmico controlado, que o ganho do FrankenPHP se manifesta sobretudo na eficiência de recursos, e não necessariamente em vazão máxima — distinção que esta pesquisa também busca capturar ao tratar RPS, latência e consumo de CPU/RAM como métricas independentes, e não como um único indicador composto.

Também relevante é o trabalho de Luz, Silva e Emer (2024), que compara experimentalmente o modelo síncrono tradicional do PHP com o modelo assíncrono viabilizado pela extensão Swoole, sob testes de carga em ambiente Docker com níveis crescentes de usuários. Diferentemente dos benchmarks puramente descritivos, os autores aplicam métodos estatísticos inferenciais — ANOVA e Teste T — para verificar se a diferença de vazão observada entre os modos é estatisticamente significativa, concluindo que o modo de operação (síncrono ou assíncrono) é o fator determinante para o desempenho (valor de $F = 9765{,}464$; $p < 2e\text{-}16$), superando em relevância estatística o próprio nível de carga aplicado. Esse resultado fornece evidência empírica adicional, específica ao ecossistema PHP, de que a persistência em memória tende a produzir ganhos de vazão estatisticamente relevantes —

corroborando a hipótese formulada na Introdução — e sugere a possível adoção de testes estatísticos inferenciais semelhantes (e não apenas comparação descritiva) na etapa de análise de resultados desta pesquisa (ver 3.5 Procedimentos de análise).

Complementarmente, benchmarks técnicos não acadêmicos também contribuem para situar as expectativas deste trabalho. Um benchmark conduzido por Byte Pursuits comparou uma aplicação PHP mínima (retornando respostas JSON estáticas) executada sob PHP-FPM e sob Swoole, utilizando a ferramenta *wrk* como injetor de carga. O autor reportou ganhos de vazão de duas a três vezes em favor do Swoole nesse cenário sintético, mas ressalvou explicitamente que se trata de "um cenário de teste pouco realista", já que uma API real dificilmente serviria apenas respostas estáticas sem qualquer chamada de I/O — operações de banco de dados tenderiam a reduzir essa vantagem relativa (BYTE PURSUITS, 2024). Essa ressalva é especialmente relevante para a discussão da validade externa do benchmark conduzido nesta pesquisa, que, ao contrário do teste de Byte Pursuits, inclui um banco de dados PostgreSQL no ambiente experimental.

Um benchmark mais recente e abrangente, publicado por Terry Lino (2026), comparou PHP-FPM, Swoole, OpenSwoole, RoadRunner e FrankenPHP servindo aplicações Laravel, avaliando latência em múltiplos percentis (p50, p90, p95, p99), vazão, consumo de memória (RSS) e uso de CPU sob diferentes níveis de concorrência. A principal conclusão do autor é que não existe um servidor de aplicação universalmente "mais rápido": diferentes servidores lideram em latência ou em vazão a depender do nível de concorrência avaliado (LINO, 2026), resultado consistente com o de Azzahidi, Wijayanto e Darmawan (2025) discutido acima.

Em conjunto, esses trabalhos correlatos indicam que, embora a persistência em memória tenda a reduzir a latência associada ao *bootstrapping* do framework e ofereça ganhos mensuráveis (inclusive estatisticamente significativos) de vazão e eficiência de recursos, o ganho efetivo depende do perfil da aplicação (com ou sem I/O), do volume de dados e do nível de concorrência — o que reforça, à luz de Jia et al. (2013), a necessidade de um benchmark específico para o CodeIgniter 4 operando sob o modo Worker do FrankenPHP, combinação ainda não coberta, até onde este levantamento bibliográfico permitiu verificar, pela literatura consultada.

3 METODOLOGIA

3.1 Caracterização da pesquisa

A pesquisa é caracterizada como experimental, quantitativa e comparativa, tendo como objeto de estudo o desempenho de diferentes arquiteturas de execução para aplicações desenvolvidas com o framework CodeIgniter 4. O caráter experimental decorre da execução de testes controlados em diferentes configurações de servidores de aplicação, enquanto a abordagem quantitativa é empregada na coleta e análise de métricas objetivas de desempenho e de utilização de recursos computacionais.

A pesquisa tem como foco a comparação entre o modelo tradicional de execução baseado em PHP-FPM e as arquiteturas proporcionadas pelo FrankenPHP, considerando seus modos Clássico e Worker. Adicionalmente, é utilizado o Laravel Octane como cenário de referência, permitindo comparar os resultados obtidos no CodeIgniter 4 com uma solução consolidada no ecossistema PHP para execução de aplicações em servidores de longa duração.

3.2 Ambiente experimental

Para garantir condições controladas e favorecer a reprodutibilidade dos experimentos, os cenários são executados utilizando contêineres Docker, com os recursos computacionais destinados a cada aplicação limitados de maneira explícita via `deploy.resources.limits` (DOCKER INC., 2024), buscando manter condições equivalentes entre os cenários avaliados.

São utilizados os seguintes cenários experimentais (Quadro 2):

Quadro 2 – Cenários experimentais e alocação de recursos

Cenário	**Descrição**	**CPU / RAM (aplicação)**
A — Legado	CodeIgniter 4 sob PHP-FPM tradicional + Nginx	1,0 CPU / 512 MB
B — Moderno Estático	CodeIgniter 4 sob FrankenPHP em modo Clássico	1,0 CPU / 512 MB
C — Vanguarda	CodeIgniter 4 sob FrankenPHP em modo Worker, com *bridge* customizada	1,0 CPU / 512 MB
D — Referência	Laravel (v. 13) sob Laravel Octane	1,0 CPU / 512 MB

Elaborado pelo autor (2026).

O injetor de carga (Grafana k6) é executado em contêiner isolado, com 2,0 CPU / 512 MB, evitando que o processamento responsável pela geração de requisições interfira nos recursos disponíveis para os servidores avaliados. O banco de dados utilizado é o PostgreSQL 16, executado em contêiner separado (0,5 CPU / 256 MB), não contabilizado no orçamento de recursos da aplicação. A aplicação é desenvolvida com CodeIgniter 4, versão 4.7.3, sob PHP 8.3.20. Antes de cada execução, é realizada a limpeza do cache do sistema operacional, de modo a reduzir a influência de um teste sobre o seguinte e preservar a comparabilidade entre os cenários — cuidado metodológico diretamente relacionado às recomendações de controle experimental discutidas por Zhan et al. (2022) e Brunst et al. (2022) na Fundamentação Teórica.

3.3 Instrumentação e geração de carga

A geração de carga é realizada por meio do Grafana k6 (GRAFANA LABS, 2024), executado de forma isolada do ambiente das aplicações, com testes conduzidos a partir de usuários virtuais (VUs), submetendo cada cenário a uma carga controlada e equivalente. A coleta de dados de consumo de recursos (CPU e RAM) é realizada por meio do subsistema de métricas do Docker.

Cada execução tem duração total de 15 minutos, distribuída da seguinte forma:

- 2 minutos de aquecimento, com 10 usuários virtuais;
- 2 minutos de rampa, elevando a carga de 10 para 100 usuários virtuais;
- 10 minutos de carga sustentada, com 100 usuários virtuais;
- 1 minuto de desaceleração, reduzindo a carga até zero.

3.4 Variáveis e métricas analisadas

O desempenho das arquiteturas é avaliado principalmente por meio das seguintes métricas, em consonância com os indicadores discutidos por Jain (1991) e disponibilizados nativamente pelo k6 (GRAFANA LABS, 2024):

- **Vazão (RPS)**: quantidade de requisições processadas por segundo pelo sistema (métrica `http_reqs` do k6);
- **Latência**: tempo necessário para processamento e resposta das requisições (métrica `http_req_duration` do k6);
- **Taxa de erro**: proporção de requisições que resultam em falha (métrica `http_req_failed` do k6);
- **Utilização de CPU**: percentual de utilização dos recursos computacionais disponíveis;

- **Utilização de memória RAM**: quantidade de memória utilizada durante a execução dos testes.

Os resultados obtidos são registrados e posteriormente organizados para permitir a comparação entre os cenários. A análise busca identificar diferenças de desempenho entre o modelo tradicional baseado em PHP-FPM, o FrankenPHP em modo Clássico, o FrankenPHP em modo Worker e o Laravel Octane.

3.5 Procedimentos de análise

Após a realização dos testes, os dados coletados são organizados e comparados entre os diferentes cenários experimentais. A análise considera a relação entre vazão, latência e utilização de recursos computacionais, identificando os cenários que apresentam maior capacidade de processamento, menor latência e melhor relação entre desempenho e recursos utilizados.

Também é considerada a complexidade técnica necessária para a utilização do FrankenPHP em modo Worker com o CodeIgniter 4, uma vez que o framework não apresenta suporte nativo a esse modelo de execução. A partir da comparação dos resultados, busca-se avaliar se a utilização de arquiteturas baseadas em persistência de memória apresenta ganhos de desempenho suficientemente relevantes para justificar sua adoção em aplicações desenvolvidas com CodeIgniter 4 — respondendo, assim, à questão norteadora e à hipótese formuladas na Introdução.

Para além da comparação descritiva das médias observadas em cada cenário, avalia-se a possibilidade de aplicar testes estatísticos inferenciais — como ANOVA e Teste T — às execuções repetidas de cada arquitetura, de modo a verificar se as diferenças de vazão e latência observadas entre os cenários são estatisticamente significativas, e não apenas resultado da variabilidade natural do experimento discutida por Laaber, Basmaci e Salza (2021). Essa abordagem segue o precedente metodológico de Luz, Silva e Emer (2024), que utilizaram os mesmos testes estatísticos para comparar os modelos síncrono e assíncrono do PHP.

REFERÊNCIAS

AZZAHIDI, Aufa Syaihan; WIJAYANTO, Bangun; DARMAWAN, Agus. **Performance Evaluation of Backend Frameworks for REST API**: A Comparative Study of Spring Boot, Flask, Express.js, Laravel FrankenPHP, and Gin. Jurnal Teknik Informatika (JUTIF), v. 6, n. 4, p. 2405-2419, ago. 2025. DOI: https://doi.org/10.52436/1.jutif.2025.6.4.4811.

BYTE PURSUITS. **Benchmarking of PHP application with php-fpm vs Swoole/OpenSwoole**. 2024. Disponível em: https://bytepursuits.com/benchmarking-of-php-application-with-php-fpm-vs-swoole-openswoole. Acesso em: 13 ago. 2026.

BRUNST, Holger et al. **First Experiences in Performance Benchmarking with the New SPEChpc 2021 Suites**. 2022. Disponível em: https://arxiv.org/abs/2203.06751. Acesso em: 13 ago. 2026.

CADDY SERVER. **Caddy 2 Documentation**. Light Code Labs, 2024. Disponível em: https://caddyserver.com/docs/. Acesso em: 13 ago. 2026.

CODEIGNITER FOUNDATION. **CodeIgniter 4 User Guide**. 2024. Disponível em: https://codeigniter.com/user_guide/. Acesso em: 13 ago. 2026.

DOCKER INC. **Compose file reference: Deploy specification (resources.limits)**. 2024. Disponível em: https://docs.docker.com/reference/compose-file/deploy/. Acesso em: 13 ago. 2026.

DUNGALAS, Kévin. **FrankenPHP: The Modern Application Server for PHP**. Les-Tilleuls.coop, 2024. Disponível em: https://frankenphp.dev/docs/. Acesso em: 13 ago. 2026.

GRAFANA LABS. **k6 Documentation: Load Testing for Engineering Teams**. 2024. Disponível em: https://grafana.com/docs/k6/latest/using-k6/metrics/. Acesso em: 13 ago. 2026.

JAIN, Raj. **The Art of Computer Systems Performance Analysis**: Techniques for Experimental Design, Measurement, Simulation, and Modeling. New York: Wiley, 1991.

JIA, Zhen et al. **The Implications of Diverse Applications and Scalable Data Sets in Benchmarking Big Data Systems**. 2013. Disponível em: https://arxiv.org/abs/1307.7943. Acesso em: 13 ago. 2026.

LAABER, Christoph; BASMACI, Mikael; SALZA, Pasquale. **Predicting unstable software benchmarks using static source code features**. Empirical Software Engineering, v. 26, art. 114, 2021. DOI: https://doi.org/10.1007/s10664-021-09996-y.

LINO, Terry. **Laravel Octane Benchmark: Swoole vs RoadRunner vs FrankenPHP**. 2026. Disponível em: https://terrylinooo.github.io/laravel-octane-benchmark/. Acesso em: 13 ago. 2026.

LOCKHART, Josh. **PHP Moderno**: Recursos atualizados para uma boa prática de desenvolvimento. São Paulo: Novatec, 2015.

LUZ, Jailson Maciel da; SILVA, Roger Sá da; EMER, Cassandro Davi. **Otimização de Aplicações Web com Programação Assíncrona em PHP e Swoole**. Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul (IFRS), Campus Avançado Veranópolis, 2024.

OPENSWOOLE. **Open Swoole Documentation: Coroutine**. 2024. Disponível em: https://openswoole.com/coroutine. Acesso em: 13 ago. 2026.

OTWELL, Taylor. **Laravel Octane Documentation**. Laravel LLC, 2024. Disponível em: https://laravel.com/docs/13.x/octane. Acesso em: 13 ago. 2026.

PHP DOCUMENTATION GROUP. **PHP: FastCGI Process Manager (FPM)**. 2024. Disponível em: https://www.php.net/manual/en/install.fpm.php. Acesso em: 13 ago. 2026.

ROADRUNNER. **RoadRunner Documentation: Workers Pool**. 2024. Disponível em: https://docs.roadrunner.dev/docs/php-worker/pool. Acesso em: 13 ago. 2026.

ZHAN, Jianfeng et al. **A BenchCouncil View on Benchmarking Emerging and Future Computing**. 2022. Disponível em: https://arxiv.org/abs/2205.07769. Acesso em: 13 ago. 2026.